Research Paper Summaries

Detailed Technical Summaries with Layers and Security Skills

Jaehyuk Lee

Contents

Overview: Skill Profile Across All Papers **2**

1 PORTAL: Fast and Secure Device Access with Arm CCA **3**

2 DeFi Security Study: User Perception of Breaches **5**

3 SENSE: Microarchitectural Event Notifications for TEEs **6**

4 autofz: Automated Fuzzer Composition at Runtime **8**

5 Hacking in Darkness: Dark-ROP against SGX Enclaves **10**

6 SGX-Bomb: Locking Down the Processor via Rowhammer **12**

7 PrivateZone: Private Execution Environment with ARM TrustZone **14**

8 OpenSGX: An Open Platform for SGX Research **16**

9 PRIME+RETOUCH: Cache Side Channel via Replacement Policy **18**

Consolidated Skill Matrix **20**

Overview: Skill Profile Across All Papers

System Layers Touched

- **Hardware / Microarchitecture:** CPU cache (L1/LLC, Tree-PLRU, dirty bits), DRAM (Rowhammer), PMU/performance counters, Memory Encryption Engine (MEE), SGX EPC/EPCM, cache replacement policy internals.
- **Firmware / TEE Runtime:** ARM TF-RMM, TrustZone Monitor mode, SGX enclave life-cycle instructions (ECREATE, EADD, EENTER, EEXIT, EGETKEY, EREPORT), SMMU page-table management.
- **Hypervisor / OS Kernel:** Linux KVM, Stage-2 page tables (IPA→PA), cgroups, perf/PMU subsystem, QEMU binary translation, Linux SGX driver.
- **Application / Protocol:** SGX enclave user-space library (`sgxlib`), Solidity smart contracts, DeFi protocol logic, fuzzer runtimes (AFL, libFuzzer, Angora, QSYM, Redqueen, etc.).

Security Skills Developed

- **TEE design & implementation:** Built/emulated Intel SGX (OpenSGX), ARM TrustZone Secure World (PrivateZone), and Arm CCA Realm isolation (PORTAL).
- **TEE exploitation:** ROP against SGX enclaves (Dark-ROP), Rowhammer DoS via MEE (SGX-Bomb), microarchitectural side-channel against enclaves (SENSE, PRIME+RETOUCH).
- **Microarchitectural side-channel attack:** Cache-timing (Prime+Probe), PLRU metadata exploitation (PRIME+RETOUCH), TSX-assisted noise-free monitoring, reverse engineering undocumented PMU on Apple M1.
- **Hardware fault injection:** Rowhammer double-sided hammering, timing-based DRAM bank probing inside an enclave.
- **DMA / peripheral isolation:** SMMU stage-2 page-table extension in TF-RMM, RMI interface between KVM and RMM.
- **Automated vulnerability discovery:** Coverage-guided fuzzing, dynamic fuzzer composition with AIMD scheduling.
- **Smart contract / DeFi security:** Longitudinal vulnerability taxonomy, user-behavior study.

1 PORTAL: Fast and Secure Device Access with Arm CCA

Venue: IEEE S&P 2025, pp. 4099–4116 [Paper Link]
Authors: Fan Sang, **Jaehyuk Lee**, Xiaokuan Zhang, Taesoo Kim (Georgia Tech / George Mason)

Background & Problem

Arm CCA (Confidential Compute Architecture) creates hardware-enforced *Realm VMs* whose memory is protected from untrusted software. Modern mobile SoCs nevertheless need Realm VMs to exchange data with DMA-capable peripherals such as GPUs, NPUs, and DSPs. Native Arm CCA uses encrypted memory for this device I/O, which imposes substantial performance and power overhead on mobile platforms.

Key Insight

PORTAL observes that the Realm Management Monitor (TF-RMM), which already enforces Realm memory access from the CPU side, can *also* manage the SMMU page tables. By migrating SMMU ownership to TF-RMM and extending the RMI call interface between KVM and RMM, Realm VMs can be granted isolated DMA regions without encryption overhead.

What Was Built

- **TF-RMM extension:** Added tracking of Realm-owned physical address (PA) ranges in RMM; extended RMM to program SMMU stage-2 page tables to enforce that DMA from peripherals can only reach designated Realm PA ranges.
- **KVM extension:** Introduced new RMI calls (`RMI_DEVICE_ATTACH`, `RMI_DEVICE_DETACH`) between the Normal World hypervisor (KVM) and RMM, allowing a Realm VM to request exclusive DMA regions.
- **SMMU driver:** Modified the Linux SMMU driver to delegate ownership of SMMU stage-2 page tables to RMM for Realm-attached devices; Normal World cannot manipulate these tables.
- **Realm guest driver:** Thin driver inside the Realm VM exposes a userspace API to request/release DMA capability from peripherals.

Layers Touched

- **Firmware (TF-RMM):** Extended enclave-equivalent monitoring to include DMA address ranges; SMMU stage-2 page table management moved into RMM.
- **Hypervisor (KVM):** New RMI calls bridge hypervisor device management and RMM.
- **Kernel Driver (SMMU):** Delegation mechanism to yield SMMU ownership to RMM.
- **Guest VM:** Realm driver to request device memory windows.

Security Skills

Arm CCA architecture (Realm/Monitor/Normal worlds), RMI call interface design, Stage-2 paging for DMA isolation, SMMU/IOMMU internals, TF-RMM firmware development, Linux KVM

modification.

2 DeFi Security Study: User Perception of Breaches

Venue: USENIX Security 2024, pp. 6039–6055 [Paper Link]
Authors: Mingyi Liu, Jun Ho Huh, HyungSeok Han, **Jaehyuk Lee**, Jihae Ahn, Frank Li, Hyoungshick Kim, Taesoo Kim

Background & Problem

Decentralized Finance (DeFi) lost $1.95 billion to scams and hacks in 2023 alone. Despite the scale of losses, no prior work had systematically investigated *user behavior*: what do DeFi users understand about security risks, and how do victims respond after being attacked?

What Was Built

A two-phase empirical study:

1. **Semi-structured interviews** ($N = 14$ DeFi users, including victims) analyzing security perception, risk tolerance, and post-incident behavior.
2. **Follow-up survey** ($N = 493$) validating interview themes at scale.

Qualitative coding of interview transcripts was performed using **Dedoose**, mapping responses to thematic categories (e.g., awareness of rug pulls, flash loans, oracle manipulation, access control bugs).

Key Findings

- Users prefer DeFi over CeFi for decentralization and high returns; they are *aware* DeFi is riskier than CeFi yet accept the risk.
- Most victims do *not* revise security practices after being attacked; instead they seek new DeFi platforms quickly — analogous to addictive gambling behavior.
- Financial incentive dominates security concern, suggesting industry-level regulation is needed, not just user education.

Layers Touched

- **Smart contract layer:** Vulnerability taxonomy including reentrancy, oracle manipulation, flash loans, access-control flaws.
- **Protocol logic:** AMMs, lending protocols, bridge contracts.
- **Research methodology:** Qualitative coding (Dedoose), survey design, statistical analysis.

Security Skills

Smart contract vulnerability taxonomy, DeFi protocol mechanics (AMMs, flash loans, bridges), qualitative interview coding (Dedoose), longitudinal security measurement, survey methodology.

3 SENSE: Microarchitectural Event Notifications for TEEs

Venue: NDSS 2024 [Paper Link]
Authors: Fan Sang, **Jaehyuk Lee**, Xiaokuan Zhang, Meng Xu, Scott Constable, Yuan Xiao, Michael Steiner, Mona Vij, Taesoo Kim

Background & Problem

Cache-based side-channel attacks (Prime+Probe, Flush+Reload) against SGX enclaves are well-studied. Existing detection defenses (e.g., TSX-based, HPC-based) fail because:

1. TEEs cannot trust the OS to relay PMU data (untrusted OS mediation).
2. Attacks can leak small amounts per execution, amortizing statistical signatures below detection thresholds.
3. TSX is platform-specific and was deprecated.

The fundamental gap is that attackers can observe microarchitectural events freely, while *victims inside TEEs cannot.*

Key Insight

Reverse the information asymmetry: expose fine-grained microarchitectural events *directly* to userspace TEEs via hardware, bypassing the untrusted OS.

What Was Built

SENSE adds three cooperating modules to the processor:

Subscription Module (SM): Each cache entry is extended with two bits: `SS` (monitored) and `SS_FAULT` (evicted while monitored). Enclave code prefetches sensitive cache lines into the *watch set* by issuing `sssub` ISA instructions. When a watched cache line is evicted, the cache sets `SS_FAULT` and piggybacks this flag on the next memory response to the CPU.

Notification Module (NM): A new CPU mode (*SENSE mode*) and ISA instructions (`ssbegin`, `ssend`, `sstramp`). Control registers `CR.SS_MODE` and `CR_SS_TRAMP` hold the trampoline address. When `SS_FAULT` is signaled, the CPU triggers a *SENSE Fault (#SS)*, clears microarchitectural state (flushes cache entries), and jumps to the userspace handler trampoline inside the enclave — *without changing privilege levels.*

Action Module (AM): Default handlers: **ABORT** (terminate on eviction), **INVARIANT** (re-prefetch evicted lines, effectively pinning them), and **THRESHOLD** (terminate after n events).

Implementation: Prototype on a cycle-accurate gem5 x86 out-of-order (O3) emulator with 4 GHz clock. Hardware overhead: ∼0.1% transistors (estimated vs. Nehalem); memory overhead: 2 bits per cache line. Performance overhead under benign workloads: **1.2%** on PolyBenchC.

Layers Touched

- **CPU microarchitecture:** Cache entry bits, cache controller logic, Load-Store Unit (LSU) memory response path.

- **ISA:** Five new instructions (`ssbegin`, `ssend`, `sstramp`, `sssub`, `ssunsub`).
- **CPU control unit:** New interrupt vector (#SS), SENSE Fault control logic.
- **SGX enclave runtime:** Userspace event handler, `_ssbegin`/`_ssend`/`_sswatch` wrappers.
- **Simulation:** gem5 O3 CPU and Classical Memory System extended with SENSE.

Security Skills

Intel SGX internals (AEX, SSA, EPCM), cache side-channel attack analysis (Prime+Probe, Flush+Reload), hardware-software co-design for TEE security, gem5 microarchitectural simulation and RTL estimation, ISA extension design, side-channel leakage quantification.

4 autofz: Automated Fuzzer Composition at Runtime

Venue: USENIX Security 2023 [Paper Link] | [GitHub]
Authors: Yu-Fu Fu, **Jaehyuk Lee**, Taesoo Kim (Georgia Tech)

Background & Problem

Dozens of diverse fuzzers exist (AFL, Redqueen, Angora, QSYM, MOpt, libFuzzer, LAF-Intel, etc.), each outperforming others on specific targets. *Selection burden*: no static analysis reliably predicts which fuzzer(s) will win on a new binary, and the rankings *invert* over time within a single campaign (rank inversion). Collaborative fuzzing (ENFUZZ, CUPID) uses static, pre-trained configurations — suboptimal when the target binary or workload phase changes.

Key Insight

Monitor runtime coverage trends (*AFL bitmap*) as a unified metric across all fuzzers, and apply an **AIMD-inspired scheduler** to reallocate CPU resources dynamically — like a gradient-following optimizer for fuzzer selection.

What Was Built

autofz (6.2K lines of Python3 + Docker) runs in two interleaved phases:

Preparation Phase: All N baseline fuzzers run in short 30-second windows with equal resources. autofz computes each fuzzer's *unique bitmap contribution* (subtracting common paths). An early-exit condition fires when the peak bitmap difference exceeds a threshold θ. θ is self-calibrated via AIMD: increase on early exit, halve otherwise — avoiding premature or overly conservative decisions.

Focus Phase: Selected fuzzers run longer with *proportional CPU allocation* based on contribution ratio. Resources are enforced via Linux **cgroups** per fuzzer process group. After Focus, seeds are synchronized across fuzzers, then Preparation restarts.

Multi-core support: Scale-up/Scale-down API per fuzzer; autofz instantiates $\lceil N \times RA[f] \rceil$ processes per fuzzer.

Evaluation (UNIFUZZ + FTS, 10 repetitions each):

- Outperforms every individual fuzzer in 11/12 UNIFUZZ and 11/12 FTS targets.
- Finds **152% more bugs** than individual fuzzers; **415% more** than CUPID/ENFUZZ.
- Average rank: 1.22 (UNIFUZZ), 1.2 (FTS) out of 11 fuzzers.

Layers Touched

- **OS scheduling:** Linux cgroups for per-fuzzer CPU quota enforcement; process pinning.
- **Fuzzer internals:** AFL bitmap (shared memory), corpus directory structures for each fuzzer, seed synchronization.
- **Coverage instrumentation:** AFL-instrumented target binary re-run to normalize bitmap across different fuzzers.

- **Container:** Docker-based deployment for reproducibility.

Security Skills

Coverage-guided fuzzing mechanics (AFL bitmap, corpus sharing), AIMD algorithm design, Linux cgroups, automated vulnerability discovery methodology, multi-fuzzer integration (AFL, AFLFast, MOpt, Redqueen, Angora, QSYM, libFuzzer, LAF-Intel, FairFuzz, Radamsa, LearnAFL).

5 Hacking in Darkness: Dark-ROP against SGX Enclaves

Venue: USENIX Security 2017 [Paper Link]
Authors: Jaehyuk Lee, Jinsoo Jang, Yeongjin Jang, Nohyun Kwak, Yeseul Choi, Changho Choi, Taesoo Kim, Marcus Peinado, Brent B. Kang

Background & Problem

Intel SGX guarantees confidentiality and integrity by encrypting all enclave memory. Even the binary running inside can be encrypted (deployed via a loader, e.g., VC3). Prior ROP attacks (including Blind ROP) assume knowledge of the target binary. The question: *can a memory-corruption vulnerability inside an SGX enclave be exploited when the attacker has zero knowledge of the enclave binary?*

Key Insight

SGX's own hardware mechanisms create oracles that reveal information about an *unknown* binary:

1. **Page-fault oracle:** AEX on page fault exposes the `CR2` register (faulting address, page-granular), allowing the attacker to determine the number of pops before a `ret` by placing non-executable page addresses on the stack.
2. **EEXIT oracle:** EEXIT does not clear registers; by chaining pop gadgets and ending with ENCLU(EEXIT), register values leak out, identifying *which* registers a gadget pops.
3. **Memory oracle:** Probing every enclave address as a potential `memcpy()` gadget — if untrusted memory is modified, the function was found.

What Was Built (Dark-ROP)

Phase 1 — Finding the vulnerability: Fuzzing exported enclave entry points and monitoring AEX page-fault signals to detect buffer-overflow conditions and locate the return address offset.

Phase 2 — Gadget discovery (from standard SGX libraries): Using the three oracles above, automatically located four gadgets from `libsgx_trts.a` and `libsgx_tstdc.a`:

1. `pop rdx; pop rcx; pop rbx; ret` + `ENCLU` (EEXIT/EREPORT/EGETKEY gate)
2. `mov rax, rbx; pop rbx; pop rbp; pop r12; ret` (rax setter)
3. `pop rsi; pop r15; ret` + `pop rdi; ret` (argument registers)
4. `memcpy()` function (arbitrary read/write between enclave and untrusted memory)

Phase 3 — Exploiting EGETKEY & EREPORT: Chained gadgets (248 bytes of ROP payload) to invoke `EGETKEY` (derive encryption key) and `EREPORT`, then exfiltrate results via `memcpy()`.

SGX Malware (MitM shadow enclave):

- Dump entire enclave binary and data to untrusted memory via `memcpy()` gadget.
- Run extracted binary outside the enclave; when SGX leaf functions are needed, invoke them via Dark-ROP calls into the real enclave.

- Hijack remote attestation: use Dark-ROP to call `EREPORT` with attacker-controlled `REPORTDATA` (containing attacker's ECDH key), then forward the signed report to the remote server.

Implementation: Intel Core i7-6700 (Skylake), Ubuntu 14.04, Intel SGX SDK.

Layers Touched

- **Intel SGX ISA:** ENCLU leaf functions (EEXIT, EGETKEY, EREPORT), AEX exception handling, TCS, SSA.
- **Memory layout:** EPC pages, enclave stack/heap, SIGSTRUCT, EINITTOKEN.
- **Exploit development:** ROP chain construction (x86-64), gadget discovery in unknown binary.
- **Remote attestation:** ECDH key exchange protocol, REPORT/Quote flow, QE interaction.

Security Skills

Intel SGX architecture (ENCLU/ENCLS, AEX, SSA, attestation), x86-64 ROP exploit construction, blind gadget discovery (oracle-based), enclave memory forensics, remote attestation protocol internals, man-in-the-middle attack construction against SGX.

6 SGX-Bomb: Locking Down the Processor via Rowhammer

Venue: SysTEX 2017 [Paper Link]
Authors: Yeongjin Jang, **Jaehyuk Lee**, Sangho Lee, Taesoo Kim

Background & Problem

Intel SGX's Memory Encryption Engine (MEE) uses an integrity tree (Merkle-tree variant) rooted in on-die SRAM to detect any tampering with EPC memory. When MEE detects an integrity violation, it applies a *drop-and-lock policy*: the memory controller is locked and the entire processor halts — requiring a cold reboot. This policy was designed to defend against *hardware* attackers who physically tamper with DRAM. The threat model of SGX explicitly does not consider software-induced DRAM bit flips because the assumption was: software cannot flip DRAM bits. The Rowhammer attack breaks this assumption.

Attack Design (Three Steps)

Step 1 — Find conflicting DRAM rows in EPC: Inside an enclave, the attacker has no physical address visibility. However, conflicting rows in the same DRAM bank show slower access latency than rows in different banks (due to row activation/precharge). The attacker scans enclave heap memory, measuring access latency of pairs (p_1, p_2) using `clflushopt` + `mfence` + load + `lfence`. Threshold: if $> 600{,}000$ cycles for 1,000 accesses, the pair is in the same bank.

Step 2 — Find interleaved (double-sided hammer targets): Because the Linux SGX driver allocates EPC pages sequentially, three consecutive conflicting virtual addresses map to sequential physical addresses. Addresses p_1 and p_3 (with p_2 in between) are used for double-sided Rowhammer against the row containing p_2.

Step 3 — Trigger bit flip and processor lockdown:

```
loop:
  mov (p1), r10; mov (p2), r11
  clflushopt (p1); clflushopt (p2)
```

After each hammering round, scan all enclave heap addresses. If any EPC page contains a flipped bit, the MEE integrity tree mismatch triggers a machine check, locking the processor.

Results on DDR4 (i7-6700K + Crucial 8GB):

- Default refresh rate (64ms): system hangs in **283 seconds**.
- Reduced refresh (128ms): 30 seconds; (256ms): 4 seconds; (503ms, max): 1 second.
- Validated: the processor ignores keystrokes, SSH, kernel interrupts (CTRL+ALT+DEL); only cold power cycle recovers.

Stealth: SGX's anti-side-channel inference (ASCI) feature hides enclave PMU cache-miss events from OS-level Rowhammer detectors (ANVIL, Linux kernel patch), making SGX-Bomb undetectable.

Layers Touched

- **DRAM hardware:** Rowhammer disturbance errors, DRAM bank mapping (timing side channel).
- **Intel SGX:** EPC memory layout, MEE integrity tree, EPCM, drop-and-lock policy, ASCI.
- **OS/driver:** Linux SGX driver sequential EPC allocation (exploited for row identification).
- **CPU fault handling:** Machine check exception pathway triggered by MEE.

Security Skills

Rowhammer attack methodology (double-sided hammering, `clflushopt`), DRAM bank mapping via timing side channel, Intel SGX memory subsystem (EPC, MEE, integrity tree), hardware fault injection to achieve DoS, understanding ASCI to evade OS-level detection.

7 PrivateZone: Private Execution Environment with ARM TrustZone

Venue: IEEE TDSC 2016 [Paper Link]
Authors: Jinsoo Jang, Changho Choi, **Jaehyuk Lee**, Nohyun Kwak, Seongman Lee, Yeseul Choi, Brent B. Kang

Background & Problem

ARM TrustZone provides a Trusted Execution Environment (TEE) for mobile devices, but it is a closed resource: only OEM-certified applications (DRM, payment) can run in the Secure World. Third-party developers cannot deploy code in the TEE without device manufacturer partnership. The goal is to open TrustZone to arbitrary developer-written Security Critical Logic (SCL) *without compromising the existing TEE services.*

Key Insight

Use ARM's virtualization extensions (Stage-2 paging) to carve out a third environment — the *Private Execution Environment (PrEE)* — that is isolated from both the Rich Execution Environment (REE) and the existing TEE. The PrEE resides in memory outside the hardware-protected TEE region (enforced by TZASC), isolated from the REE by Stage-2 page tables managed by PrivateZone in Monitor mode.

Architecture

Three environments: REE (Normal World, Linux/Android), TEE (Secure World, SierraTEE), PrEE (new, software-isolated).

Stage-2 paging: At boot, PrivateZone creates two Stage-2 page tables — one for the REE (1.8 GB) and one for the PrEE (201 MB) — in the bootloader. VTTBR register switches which page table is active. Neither environment can access the other's physical memory. TEE is hardware-isolated from both via TZASC.

Environment switching (Monitor mode): All switches go through PrivateZone in Monitor mode, triggered by `SMC` instructions. SCL invocation is triggered via `SVC #0xdeadbf` in user mode, handled in the REE kernel, which calls `SMC`.

SCL lifecycle: Deploy: PrivateZone copies SCL from REE to PrEE, computes SHA1 hash, verifies against developer-signed hash. **Invoke**: PrivateZone saves REE context, copies parameters from REE to PrEE (with validation), switches VTTBR to PrEE, jumps to SCL entry. **Revoke**: clears SCL pages.

Security: SCL pages have PXN (Privileged Execute Never) and WXN flags; kernel objects and SCL pages are never on the same physical page (prevents ret2dir). IRQ forwarded to REE for scheduling; register state cleared before world switch.

Implementation: Arndale board (Cortex-A15 dual-core, 1.7 GHz), Linaro-Android 13.11, SierraTEE. 4,627 lines of C in TEE (Monitor mode); 340 lines of ARM assembly in PrEE.

Overhead: ∼3% on Phoronix application benchmarks; SCL invocation adds ∼3.3× overhead dominated by SHA1 hash check during deployment (2440 µs).

Layers Touched

- **ARM TrustZone:** Monitor mode (highest privilege), Secure Configuration Register (SCR), TZASC.
- **Stage-2 paging (virtualization extension):** VTTBR, HCR, IPA→PA translation tables created at boot.
- **Normal World OS (Linux):** Kernel driver for `SMC` invocation; bootloader memory partitioning.
- **PrEE kernel:** Minimal exception handlers (SVC, prefetch abort, IRQ, data abort) in ARM assembly.

Security Skills

ARM TrustZone architecture (Monitor mode, world switch, SCR/VTTBR/HCR registers), Stage-2 (IPA→PA) page table construction for software isolation, PXN/WXN security flags, Secure World OS design, TZASC configuration, cache maintenance across worlds (`DCIMVAC`), ARM assembly coding.

8 OpenSGX: An Open Platform for SGX Research

Venue: NDSS 2016 [Paper Link]
Authors: Prerit Jain, Soham Desai, Seongmin Kim, Ming-Wei Shih, **JaeHyuk Lee**, Changho Choi, Youjung Shin, Taesoo Kim, Brent B. Kang, Dongsu Han

Background & Problem

Intel SGX was announced before actual hardware was available; the first SGX CPUs shipped in October 2015 on a handful of Dell laptops. The research community had no way to build, test, or study SGX enclaves. Intel's own emulator was available only to a closed group of collaborators. OpenSGX was the first publicly available, instruction-level SGX emulator.

What Was Built (19K lines of C)

OpenSGX extends QEMU's usermode binary translation (TCG) with six components:

1. Hardware Emulation (5,954 lines): All SGX ISA instructions (Table I in paper: ECREATE, EADD, EEXTEND, EINIT, EENTER, EEXIT, ERESUME, EGETKEY, EREPORT, EAUG, EACCEPT, etc.) implemented as QEMU helper routines. CPU state extended with `CREGS` (enclave mode flag `CR_CURR_EIP`, etc.). EPC emulated as a contiguous virtual memory region; EPCM maintained in software. All memory accesses instrumented: `gen_helper_mem_access()` checks source (enclave/non-enclave), destination (EPC/non-EPC), and EPCM permissions.

2. OS Emulation Layer (1,531 lines): New system calls (`sys_sgx_init`, `sys_create_enclave`, `sys_add_epc`, `sys_stat_enclave`) service ENCLS (privileged) instructions. Enclave initialization sequence: ECREATE $\rightarrow$ EADD $\times$ N $\rightarrow$ EEXTEND $\times$ $(N \times 16)$ $\rightarrow$ EINIT.

3. Enclave Loader: Reads ELF sections (`.enc_text`, `.enc_data`) and loads them into EPC pages. Custom linker script ensures PIC-compatible enclave binaries.

4. `sgxlib` User Library (2,978 lines): APIs for both host (`sgx_enter`, `sgx_host_read/write`) and enclave (`sgx_exit`, `sgx_getkey`, `sgx_getreport`, `sgx_malloc`, `sgx_memcpy`). Communication via **trampoline** (code) and **stub** (data) shared regions. `sgx_malloc()` uses EAUG/EACCEPT for dynamic EPC allocation; EACCEPT validates the new page's EPCM pending bit to thwart Iago attacks.

5. GDB Debugger Support: GDB stub maps translated QEMU instructions back to original source; new commands: `info epc`, `info epcm`, `info secs`, `list enclaves`.

6. Performance Monitoring: Per-enclave descriptor counts context switches, ENCLU/ENCLS instructions, TLB flushes, dynamically allocated EPC pages.

Case study: Ported Tor directory server and exit node to OpenSGX, protecting identity/signing keys inside enclaves; measured 3.2×–12× CPU cycle overhead vs. native QEMU (dominated by `sgx_send`/`sgx_recv` calls crossing enclave boundaries).

Layers Touched

- **CPU emulation (QEMU):** Binary translation (TCG) extended with SGX opcode dispatch; per-instruction memory access instrumentation.

- **SGX ISA:** All enclave lifecycle instructions, all data structures (SECS, TCS, SSA, EPCM, SIGSTRUCT, EINITTOKEN, KEYREQUEST, REPORT).
- **OS/system calls:** New Linux-like syscall interface for enclave management.
- **User-space runtime:** `sgxlib` enclave library; trampoline/stub communication protocol.
- **Debug tooling:** GDB Python extension for SGX-specific state inspection.

Security Skills

Intel SGX ISA and data structures (ECREATE through ERESUME, EPCM, SECS, TCS, SSA), QEMU binary translation internals and extension, enclave memory management, Iago attack mitigation via EAUG/EACCEPT, remote attestation protocol (EREPORT/EGETKEY), GDB extension development, systems programming for TEE platform design.

9 PRIME+RETOUCH: Cache Side Channel via Replacement Policy

Venue: arXiv:2402.15425 [Paper Link]
Authors: Jaehyuk Lee, Fan Sang, Taesoo Kim (Georgia Tech)
Acknowledged by Intel.

Background & Problem

Prefetching-and-locking defenses (e.g., Cloak using Intel TSX) claim to prevent cache side-channel leakage by preloading sensitive data into the cache and locking it with a TSX transaction. If the attacker evicts a protected cache line, the TSX transaction aborts and the victim detects the attack. The core assumption: *meaningful information leaks only through cache evictions.*

PRIME+RETOUCH breaks this assumption: information leaks through the *Tree-PLRU replacement policy metadata* without ever evicting the victim's data.

Key Technical Contribution: Leaky Tree-PLRU

The Tree-PLRU policy uses a binary tree per cache set where intermediate nodes store bits indicating which sub-tree was accessed less recently. When a cache line is accessed, all flags on the path root→leaf are flipped to mark it as MRU.

Critical observation: Back-to-back accesses to the same cache line (prefetch then genuine access) are *indistinguishable* in the final tree state — both result in the same PLRU metadata. PRIME+RETOUCH makes them distinguishable by inserting an attacker "retouch" between the prefetch and the genuine access:

1. **Prime:** Attacker primes all ways of the target cache set.
2. **Wait:** Victim prefetches sensitive data V into cache.
3. **Retouch:** Attacker accesses the current PLRU cache way (which was occupied by V's prefetch) using *PLRU Aware Retouch* — always targeting the cache way that the Tree-PLRU marks as next eviction candidate.
4. **Probe:** Force one cache miss; observe which way is evicted — reveals whether V was accessed after the prefetch.

PLRU Aware Retouch: Standard retouch suffers false positives when the attacker retouches before the victim prefetches (unsynchronized). The solution: always retouch the *post-prime PLRU entry* (known to the attacker from the Tree-PLRU state), which deterministically resolves synchronization ambiguity. This reduces unsynchronized sequences to synchronized equivalents — allowing off-line analysis of tree state after the attack, rather than real-time timing.

Poor man's algorithm (false-negative elimination): When precise synchronization is unavailable, autofz-style delay reduction identifies the exact window between victim prefetch and genuine access.

Reverse Engineering the Apple M1 L1 Cache

- No public documentation for M1 cache architecture.

- Discovered undocumented PMU event `0xa3` (L1 D-cache miss) and `0x2` (core clock cycles) via XNU kernel source and Rosetta2 binary translation of `rdtsc`.
- Read `CCSIDREL1`/`CSSELREL1` system registers to retrieve cache hierarchy (Firestorm: 128KB, 256 sets, 8-way; Icestorm: 64KB, 128 sets, 8-way).
- Confirmed Tree-PLRU on M1 L1 cache using timing differences (L1 hit: ∼2–4 cycles; L1 miss: ∼12–16 cycles).

M1 challenge: System timer (`s34c15c106`) is too coarse (1MHz–50MHz vs. 3.2GHz core clock) to distinguish L1 hit/miss. Solution: accumulate 14 additional cache miss events using a crafted access sequence that causes misses *only* when the victim accessed data. The amplified latency becomes measurable with the coarse timer.

Attacks Demonstrated

- **AES T-Table (x86):** Leaks T-Table access patterns with prefetch-and-locking (Cloak/TSX) deployed; achieves same accuracy as classic Prime+Probe without defense. Only 0.2% of executions cause TSX faults (vs. 93.4% for Prime+Probe).
- **AES T-Table in SGX enclave (x86):** Identical leakage inside SGX enclaves with prefetch-based defense.
- **Apple M1:** Demonstrated feasibility of leaking victim access pattern without fine-grained timer.

Layers Touched

- **L1 cache microarchitecture:** Tree-PLRU binary tree state, 8-way set-associative L1 D-cache.
- **Intel TSX:** Noise-free reverse engineering using write-set vs. read-set eviction behavior.
- **Apple M1 PMU:** Undocumented PMU interface, system register access at EL0.
- **Intel SGX (bonus):** Attack works within SGX-protected enclaves with TSX-based cache locking defense.

Security Skills

Tree-PLRU replacement policy reverse engineering (Intel Coffee Lake, Apple M1), noise-free cache monitoring via Intel TSX, eviction policy metadata exploitation, undocumented PMU interface discovery on Apple M1 (via Rosetta2 analysis), signal amplification for coarse-timer platforms, bypassing prefetch-and-locking cache side-channel defenses (Cloak), ARM64 assembly and pointer chasing with memory barriers.

Consolidated Skill Matrix

Skill Area	Papers / Evidence
TEE design & implementation	OpenSGX (SGX emulation), PrivateZone (TrustZone PrEE), PORTAL (CCA Realm + SMMU)
TEE exploitation	Dark-ROP (ROP via SGX oracles), SGX-Bomb (Rowhammer+MEE DoS), SENSE (side-channel in enclave)
Microarch. side-channel attack	SENSE (cache eviction notifications), PRIME+RETOUCH (PLRU metadata leak)
Cache replacement policy	PRIME+RETOUCH (Tree-PLRU reverse engineering, Intel + Apple M1)
Hardware fault attacks	SGX-Bomb (Rowhammer double-sided + MEE integrity fault)
DMA & peripheral isolation	PORTAL (SMMU stage-2 tables in TF-RMM, RMI design)
Hypervisor & Stage-2 paging	PORTAL (KVM + RMM), PrivateZone (Stage-2 IPA→PA)
Firmware development	PORTAL (TF-RMM extension), PrivateZone (Monitor mode ARM asm)
SGX ISA & enclave lifecycle	OpenSGX (QEMU emulation), Dark-ROP (ENCLU oracle exploitation)
Fuzzing & vuln. discovery	autofz (AIMD meta-fuzzer, cgroups, 11 fuzzers)
Smart contract / DeFi security	DeFi study (vulnerability taxonomy, user behavior, Dedoose)
Emulation & tooling	OpenSGX (QEMU + sgxlib + GDB extension)
ROP exploit construction	Dark-ROP (blind ROP against unknown encrypted binary)